

TITANS

Titans — 이름 출처, 정확 구현, 정직한 검증 결과

이름 출처 (제가 만든 게 아님)

- 설계 문서(사용자 제공) §3.1의 `base_rule` 후보에 `titans` 가 이미 있었음.
- 원 출처: Google Research 논문 "Titans: Learning to Memorize at Test Time" (Behrouz, Zhong, Mirrokni — arXiv:2501.00663, 2025). 어텐션 + 테스트시점 학습 신경 장기기억.

논문 핵심 식

$$\begin{aligned} \ell(W; x_t) &= ||W \cdot k_t - v_t||^2 && \text{(연관 손실; } k=xW_K, v=xW_V, q=xW_Q) \\ \nabla \ell &= (W \cdot k_t - v_t) \cdot k_t^T && \text{(행렬 메모리 기울기)} \\ S_t &= \eta_t \cdot S_{\{t-1\}} - \theta_t \cdot \nabla \ell && \text{(모멘텀: 과거+현재 surprise)} \\ W_t &= (1 - \alpha_t) \cdot W_{\{t-1\}} + S_t && \text{(\alpha_t: 데이터 의존 망각/weight-decay 게이트)} \\ y_t &= W_t \cdot q_t && \text{(현재 메모리로 읽기)} \end{aligned}$$

η_t 모멘텀, θ_t 학습률, α_t 망각(모두 데이터 의존). 추론 상태 $(W,S)=O(d^2)$ 상수 $\rightarrow$ 상수메모리 추론.

세 가지 "titans" 구현과 실측 (정직)

구현	무엇	MQAR recall(vocab32/pairs2/L64, 4090)
<code>base_rule="titans"</code> (model.py)	dla 감쇠 + 정적 영속읽기 (근사)	부분~1.0(grok, 학습 많이) — 단 논문 메커니즘 아님
<code>titans_exact.py</code> (우리 from-scratch)	논문 식 직접 재현	0.06 = chance (학습 실패) ⚠
lucidrains <code>titans-pytorch</code>	검증된 정확 구현	0.98 ✅

$\rightarrow$ 결론: 자체 정확 구현(titans_exact)은 chance로 실패. 안정화(키/쿼리 L2정규화+출력 norm)로 NaN(0.0) $\rightarrow$ chance(0.06)까지 갔으나 학습은 안 됨(청크병렬·정확기울기·메모리MLP 등 미충족). 정확한 Titans는 lucidrains `titans-pytorch` 를 채택(`growing_memory.titans_backend`).

사용 (정확 구현)

```
pip install "growing-memory-pytorch[titans]" # titans-pytorch 포함
```

```
from growing_memory.titans_backend import neural_memory, available
assert available()
mem = neural_memory(dim=384, chunk_size=64)      # lucidrains NeuralMemory
retrieved, state = mem(seq)                     # [B,L,d]
```

AutoResearch 통합 (base_rule="titans_real")

검증된 lucidrains NeuralMemory를 노드의 믹서로 연결 — `real/model.py` 의 MemoryMixer가 `base_rule="titans_real"` 이면 `titans_pytorch.NeuralMemory` 를 사용(aggregation 우회). `grok_tune` VARIANTS에 `titans_real` 추가 → 스왑/학습에 진짜 Titans 포함.

recall 차이의 진짜 원인 — 통합 버그(chunk_size), 난이도 아님 (정직 정정)

처음엔 "통합 0.33 vs 교차검증 0.98"을 난이도 차이로 설명했으나 틀렸다. 변수 격리로 규명:

1. 같은 쉬운 조건(seq=64/pairs2)인데 하니스만 바뀌도 0.98→0.32 → 난이도 아님, 스캐폴드/통합 문제.
2. weight tying 해제 → 무효(0.31→0.31). 진짜 원인은 lucidrains `chunk_size` :

chunk_size (seq=64)	recall(3k)
16	0.521
32	0.520
64 (내 통합 버그)	0.314

내 통합이 `chunk_size=max(16, segment_len)=64` 로 뒤서 `seq=64`면 전체가 1청크 → 메모리 갱신 1회뿐 → 회상 붕괴. 수정: `chunk_size=min(32, segment_len)` (시퀀스 내 여러 번 갱신).

수정 후 본 런(seq=64/pairs2, 8k step): recall 0.31 → 0.55 (수정 효과 확인). 단 8k에서도 grok 점프 없이 0.55 정체 — 우리 GrowingMemoryModel 스캐폴드는 grok이 늦다(우리 titans도 13~15k 걸림; titans_compare 스캐폴드는 chunk32에서 3k에 0.98 grok). 남은 차이는 RMSNorm vs LayerNorm + SSL front(weight tying은 배제됨). → 0.9+는 (a) 더 많은 스텝(~15~20k) 또는 (b) 스캐폴드 정렬 필요. 정직: 본 런은 0.9+ 미달(0.55), 모듈 자체는 vetted 스캐폴드에서 0.98 확인됨.

→ 교훈: 통합 시 라이브러리 하이퍼파라미터(chunk_size)를 잘못 매핑하면 성능이 무너진다. 실행:

`~/gm_venv/bin/python grok_tune.py --chunked --variants titans_real ...` (titans-pytorch 필요).

교훈

사용자 지적("titans는 네가 만든 거냐?")이 정확했다. 이름은 Google 논문 차용, 자체 구현은 근사/실패였다. "정확 구현을 하거나 검색하거나" 중 검색·채택(lucidrains)이 정답이었음. 재현: `packages/growing-memory/titans_compare.py`.